UT 2 – GUI with Jetpack Compose

Block 2 · Layouts: Column, Row and Box

Interface Development (0488) · 2nd DAM · 2026/2027

In this block

In Block 1 your screens had a single element. A real screen has many: a header, rows, cards, lists. In this block you learn the **layouts** of Compose, the containers that place several elements on the screen, and the modifiers you need to give them size, space, colour and shape. With them you will be able to build Exercise 1 of the exercises project: a screen with a header, boxes of equal width, and a scrollable list of cards generated from data.

By the end of this block you will be able to:

- place elements vertically, horizontally and on top of each other with `Column`, `Row` and `Box`;
- control the space between elements and their position with `Arrangement` and `Alignment`;
- set sizes and share the space of a row with `weight`;
- give colour and shape to a composable with `background`, `clip` and `border`;
- make a screen scroll vertically;
- split a screen into small composables with parameters;
- repeat a composable for each item of a list of data.

What you need: the knowledge of Block 1 (composables, `Text`, modifiers, previews) and the basics of lists and `data class` from UT 1.

Contenido

In this block 1
2.1 From one element to many 3
2.2 Column 4
2.3 Row 4
2.4 Box 5
2.5 Main axis and cross axis 6
Values of Arrangement 6
Values of Alignment 6
2.6 Space and size 8
Padding and spacing 8
Size 9
Sharing the space with weight 9
Spacer 9
2.7 Combining layouts 10

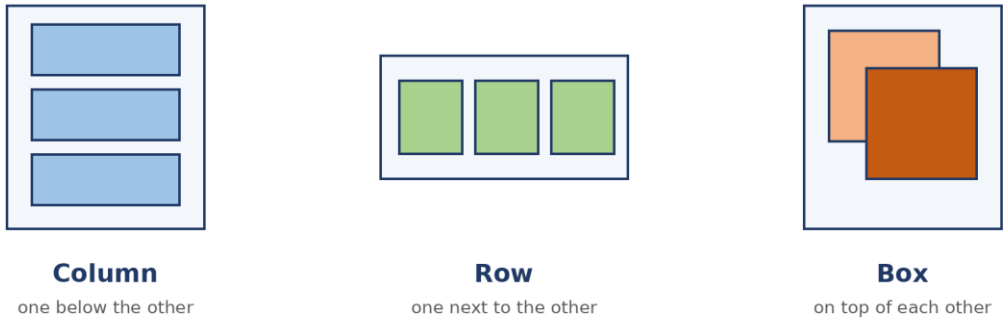
2.8 Colour and shape 12
Colours 12
Background, shape and border 12
Text on a coloured background 13
2.9 Scrolling 13
2.10 Splitting the screen into composables 14
2.11 Repeating UI from data 15
2.12 From design to code 15
2.13 Working in the exercises project 16
2.14 Further practice and reading 16
More practice 16
Documentation 17
Self-check questions 17
Key terms 17

2.1 From one element to many

A composable emits UI, but if you write two `Text` one after the other inside a function, Compose does not know how to place them: they would be drawn on top of each other. You must put them inside a **layout**, a composable whose job is to place its children.

Compose has three basic layouts:

Layout	How it places its children	Think of it as
`Column`	One below the other (vertical)	A list of lines
`Row`	One next to the other (horizontal)	A line of items
`Box`	On top of each other (stacked)	A pile of sheets



The three basic layouts

If you know XML: `Column` is like a vertical `LinearLayout`, `Row` is like a horizontal one, and `Box` is like a `FrameLayout`.

Layouts can contain other layouts. A screen is a **tree**: the layout at the top contains rows, boxes and columns, and these contain the final elements (`Text`, images...). Most designs are solved by combining the three layouts.

```
Column                    // the whole screen
|-- Text                   // title
|-- Row                    // three items side by side
|   |-- Box
|   |-- Box
|   \-- Box
\-- Text                   // footer
```

> **Key idea**
> Before writing code, look at the design and ask: **what is vertical, what is horizontal, what is stacked?** The answer is the tree of layouts.

2.2 Column

`Column` places its children one below the other. Its content goes inside the braces, like `setContent`:

```
@Composable
fun ProfileHeader(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier.fillMaxWidth().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Ana García", fontSize = 22.sp, fontWeight = FontWeight.Bold)
        Text("Student of 2nd DAM")
        Text("Villarrobledo")
    }
}
```

A `Column` has two parameters that control its children (see section 2.5):

- `verticalArrangement`: how the children are **distributed** along the vertical direction, and the space between them.
- `horizontalAlignment`: where the children are **aligned** horizontally (left, centre, right).

By default a `Column` is only as big as its content (it adapts to what it contains), and its children are aligned to the start (left). To make it fill the screen, add `fillMaxSize()` to its modifier. For example, to distribute five texts over the whole height of the screen:

```
Column(
    modifier = Modifier.fillMaxSize(),
    verticalArrangement = Arrangement.SpaceAround
) {
    repeat(5) { number -> Text("Hello ${number + 1}") }
}
```

If there are more elements than the screen can show, the column needs a scroll (section 2.9).

2.3 Row

`Row` is the same idea in the horizontal direction. The parameters change their names:

```
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween,
    verticalAlignment = Alignment.CenterVertically
) {
    Text("Home")
    Text("Search")
    Text("Profile")
}
```

- `horizontalArrangement`: how the children are distributed along the row.
- `verticalAlignment`: where the children are aligned vertically (top, centre, bottom). If the children have different heights, `Alignment.CenterVertically` is the one you normally want.

Everything you learn about `Column` works in the same way for `Row`, changing vertical for horizontal. If the elements do not fit in the width of the screen, you can add a **horizontal scroll**:

```
Row(
    modifier = Modifier
        .fillMaxWidth()
        .horizontalScroll(rememberScrollState())
) {
    repeat(10) { number ->
        Text("Item ${number + 1}", modifier = Modifier.padding(8.dp))
    }
}
```

2.4 Box

`Box` stacks its children: the first one is drawn first, and the following ones are drawn **on top**. It is used to place something over something else, and also to centre a single element inside a space.

```
// Centre one element
Box(
    modifier = Modifier.size(120.dp).background(Color.LightGray),
    contentAlignment = Alignment.Center
) {
    Text("Centre")
}

// Stack two elements and place each one in a corner
Box(modifier = Modifier.size(120.dp).background(Color.LightGray)) {
    Text("Top start")
    Text("Bottom end", modifier = Modifier.align(Alignment.BottomEnd))
}
```

`contentAlignment` positions **all** the children of the box. If you need a different position for one child, give that child `Modifier.align(...)`. By default the children go to the top-left corner (`Alignment.TopStart`).

Two details about `Box`:

- Without size modifiers, a `Box` is only as big as its content. A `Box` does not cut its content by default; if you want to cut whatever goes outside the box, add `Modifier.clipToBounds()`.
- `contentAlignment` positions the **direct children** of that box. If a child is another `Box`, it positions that box, not what is inside it. To centre the text of the inner box too, you must use `contentAlignment` again in the inner box:

```
// The outer box centres the red box
Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
    Box(
        modifier = Modifier.size(200.dp).background(Color.Red),
        contentAlignment = Alignment.Center   // this one centres the text
    ) {
        Text("Hello")
    }
}
```

Exercise 2.1

In a new file of your `UT2_Compose` project (for example in the package `exercises.layouts`), create three previews: a `Column` with three texts, a `Row` of three texts distributed with `SpaceBetween`, and a `Box` with two overlapping texts. Change the `Arrangement` and `Alignment` parameters and observe what moves.

2.5 Main axis and cross axis

Each layout has a **main axis**, the direction in which it places its children, and a **cross axis**, the perpendicular one. Compose uses two different words for them:

	Main axis → `Arrangement`	Cross axis → `Alignment`
`Column`	`verticalArrangement`	`horizontalAlignment`
`Row`	`horizontalArrangement`	`verticalAlignment`

An easy way to remember it: **Arrangement** is how the children are arranged along the direction of the layout; **Alignment** is how they are aligned across it.

Values of Arrangement

Value	Effect
`Top`, `Center`, `Bottom` (Column) `Start`, `Center`, `End` (Row)	Groups the children at one end or in the centre.
`SpaceBetween`	First and last child at the ends, equal space between the others.
`SpaceAround`	Equal space around each child (half of it at the ends).
`SpaceEvenly`	Equal space between children and at the ends.
`spacedBy(8.dp)`	A fixed space between children. The children stay grouped at the start.

Values of Alignment

Layout	Values
`Column` (`horizontalAlignment`)	`Start`, `CenterHorizontally`, `End`
`Row` (`verticalAlignment`)	`Top`, `CenterVertically`, `Bottom`
`Box` (`contentAlignment`)	`TopStart`, `TopCenter`, `TopEnd`, `CenterStart`, `Center`, `CenterEnd`, `BottomStart`, `BottomCenter`, `BottomEnd`

Arrangement only has an effect when the layout has **free space** to distribute. A `Column` that is only as big as its content has no free space: use `fillMaxSize()` or `fillMaxWidth()` in its modifier if you want the distribution to be visible.

> **Exercise 2.1b – Arrangement and Alignment lab**
> Copy this code into a new file. The three pieces have different sizes on purpose, so that you can see the effect of each parameter. **Before you look at each preview, predict where the pieces will be.** Then compare your prediction with the result.

Needs the imports of Box, Column, Row, size, width, height, background, Dp, Color, etc.

```
@Composable
fun Piece(label: String, width: Dp, height: Dp, color: Color) {
    Box(
        modifier = Modifier.width(width).height(height).background(color),
        contentAlignment = Alignment.Center
    ) { Text(label, color = Color.White) }
}

@Composable
fun ColumnLab(arrangement: Arrangement.Vertical, alignment: Alignment.Horizontal) {
    Column(
        modifier = Modifier.size(300.dp).background(Color.LightGray),
        verticalArrangement = arrangement,
        horizontalAlignment = alignment
    ) {
        Piece("A", 80.dp, 50.dp, Color(0xFFE53935))
        Piece("B", 140.dp, 50.dp, Color(0xFF43A047))
        Piece("C", 200.dp, 50.dp, Color(0xFF1E88E5))
    }
}

@Composable
fun RowLab(arrangement: Arrangement.Horizontal, alignment: Alignment.Vertical) {
    Row(
        modifier = Modifier.size(300.dp).background(Color.LightGray),
        horizontalArrangement = arrangement,
        verticalAlignment = alignment
    ) {
        Piece("A", 60.dp, 50.dp, Color(0xFFE53935))
        Piece("B", 60.dp, 100.dp, Color(0xFF43A047))
        Piece("C", 60.dp, 150.dp, Color(0xFF1E88E5))
    }
}

@Preview(name = "1 Default", showBackground = true)
@Composable
fun ColumnLab1() = ColumnLab(Arrangement.Top, Alignment.Start)

@Preview(name = "2 Arrangement only", showBackground = true)
@Composable
fun ColumnLab2() = ColumnLab(Arrangement.Center, Alignment.Start)

@Preview(name = "3 Alignment only", showBackground = true)
@Composable
fun ColumnLab3() = ColumnLab(Arrangement.Top, Alignment.CenterHorizontally)
```

```
@Preview(name = "4 Both", showBackground = true)
@Composable
fun ColumnLab4() = ColumnLab(Arrangement.SpaceBetween, Alignment.End)

@Preview(name = "5 spacedBy", showBackground = true)
@Composable
fun ColumnLab5() =
    ColumnLab(Arrangement.spacedBy(20.dp), Alignment.CenterHorizontally)

@Preview(name = "6 Row default", showBackground = true)
@Composable
fun RowLab1() = RowLab(Arrangement.Start, Alignment.Top)

@Preview(name = "7 Row: both", showBackground = true)
@Composable
fun RowLab2() = RowLab(Arrangement.SpaceEvenly, Alignment.CenterVertically)
```

Preview	What you should see
1	The three pieces at the top-left corner: `Top` and `Start`.
2	The pieces move to the vertical centre, but stay on the left: `Arrangement` only moved them along the column.
3	The pieces stay at the top, but each one is centred horizontally: `Alignment` only moved them across the column.
4	One piece at the top, one at the bottom and one in between, all on the right: both parameters work at the same time.
5	Fixed gaps of `20.dp` between the pieces, grouped at the top: `spacedBy` separates, it does not distribute the free space.
6	The pieces at the top-left corner of the row.
7	Equal gaps between the pieces, and the three centred vertically: in a `Row` the two axes are swapped.

What do you conclude? **Each parameter acts on one axis and does not affect the other.** Try other values from the tables above and, before running, predict the result.

2.6 Space and size

Padding and spacing

There are two ways of creating space, and they do different things:

- `padding` adds space **around** a composable (or inside its background, depending on the order, as you saw in Block 1).
- `Arrangement.spacedBy(...)` adds space **between** the children of a layout. It is the cleanest way of separating the elements of a list, because you write it once instead of adding padding to every child.

```
Modifier.padding(16.dp)                          // all four sides
Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
Modifier.padding(start = 4.dp, top = 6.dp)       // only some sides
```

Size

Modifier	Effect
`fillMaxWidth()`	Takes all the available width.
`fillMaxSize()`	Takes all the available width and height.
`size(56.dp)`	Fixed width and height (a square).
`width(100.dp)`, `height(90.dp)`	Fixed width or height.

Sharing the space with weight

Sometimes you want several children to share the width of a row without calculating pixels. `Modifier.weight(...)` does it: the children with weight share the **remaining** space in proportion to their weights.

```
// Three boxes of the same width
Row(modifier = Modifier.fillMaxWidth()) {
    Box(Modifier.weight(1f).height(60.dp).background(Color.Red))
    Box(Modifier.weight(1f).height(60.dp).background(Color.Green))
    Box(Modifier.weight(1f).height(60.dp).background(Color.Blue))
}

// With weights 1 and 2, the second takes twice the width of the first
```

Notes about `weight`:

- It can only be used on a **direct child** of a `Row` or a `Column`: in a `Row` it shares the width, in a `Column` the height.
- Do not combine `weight` with a fixed `width` in the same direction.
- Do not use vertical `weight` inside a column that scrolls (section 2.9): a scrolling column has no limit of height to share.

Exercise 2.2

Create a `Row` with three coloured `Box` of height `80.dp`. First make them equal with `weight(1f)`. Then use weights 1, 1 and 2. Finally replace the weights with `width(100.dp)` in a narrow preview (`@Preview(widthDp = 250)`) and see what happens when the boxes do not fit. What do you conclude about `weight` and fixed widths?

Spacer

`Spacer` is an empty composable that only takes space. It is useful when the gaps between elements are **not all equal**, so `spacedBy` is not enough:

```
Column {
    Text("Title")
    Spacer(modifier = Modifier.height(24.dp))   // a bigger gap here
    Text("First paragraph")
    Spacer(modifier = Modifier.height(8.dp))
    Text("Second paragraph")
}

Row(modifier = Modifier.fillMaxWidth()) {
    Text("Title")
    Spacer(modifier = Modifier.weight(1f))      // pushes "Edit" to the end
    Text("Edit")
}
```

You can create the same gap with `padding`, but `Spacer` says clearly what you mean. Use `height` in a `Column` and `width` in a `Row`; with `weight(1f)` the spacer takes all the free space. As with every modifier chain, the order matters: `padding` before `background` leaves the space outside the colour, and after it leaves the space inside.

2.7 Combining layouts

Complex screens are built by **nesting** the three layouts. Look at this screen: it is divided into three areas of equal height, and the middle one contains a row with two boxes.

```
Column                          // the whole screen
|-- Box (RED)                   // weight 1
|-- Box (BLUE)                  // weight 1
|   \-- Row                     // inside the blue box
|       |-- Box (GREY)
|       \-- Box (WHITE)
\-- Box (YELLOW)                 // weight 1
```

Needs the imports of fillMaxHeight, background, Color, etc.

```
@Composable
fun ColourAreas(modifier: Modifier = Modifier) {
    Column(modifier = modifier.fillMaxSize()) {
        Box(
            modifier = Modifier
                .weight(1f)
                .fillMaxWidth()
                .background(Color.Red),
            contentAlignment = Alignment.Center
        ) { Text("RED") }

        Box(
            modifier = Modifier
                .weight(1f)
                .fillMaxWidth()
                .background(Color.Blue)
        ) {
            Row(modifier = Modifier.fillMaxWidth().height(40.dp)) {
                Box(
                    modifier = Modifier
                        .weight(1f)
                        .fillMaxHeight()
```

```
                        .background(Color.Gray),
                    contentAlignment = Alignment.Center
                ) { Text("GREY") }
                Box(
                    modifier = Modifier
                        .weight(1f)
                        .fillMaxHeight()
                        .background(Color.White),
                    contentAlignment = Alignment.Center
                ) { Text("WHITE") }
            }
        }

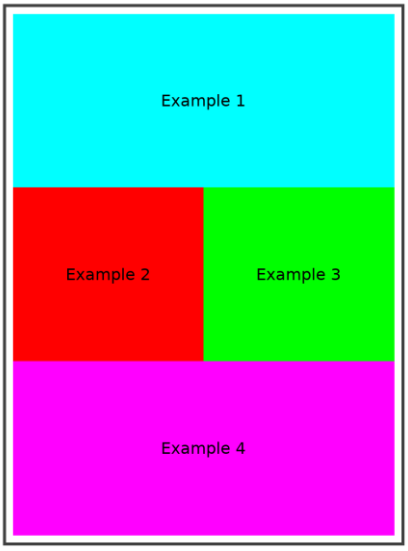
        Box(
            modifier = Modifier
                .weight(1f)
                .fillMaxWidth()
                .background(Color.Yellow),
            contentAlignment = Alignment.Center
        ) { Text("YELLOW") }
    }
}
```

Read it as a tree and look at the decisions:

- The three `weight(1f)` live in a `Column`, so they share the **height** equally. The two `weight(1f)` of the inner `Row` share the **width**.
- Each box that contains a text has its **own** `contentAlignment`, as you learnt in section 2.4.
- The blue box has no `contentAlignment`, so its `Row` is placed at the top-left corner, and the `Row` only takes `40.dp` of height.

Exercise 2.4

Build this screen: a `Column` that fills the whole screen with three areas of equal height. The top area is cyan, with the text “Example 1”. The middle area is a `Row` with two boxes of equal width: red (“Example 2”) and green (“Example 3”). The bottom area is magenta, with the text “Example 4”. All the texts are centred.



Design for Exercise 2.4

2.8 Colour and shape

Colours

Compose has some predefined colours (`Color.White`, `Color.Black`, `Color.Red`, `Color.LightGray`...). To use any other colour you write its code in hexadecimal:

```
Color(0xFF2196F3)      // a blue
Color(0x802196F3)      // the same blue, half transparent
Color(0xFF2196F3).copy(alpha = 0.5f)   // another way to get the same
```

The code has eight digits: **AA RR GG BB**. The first two are the transparency (`FF` is fully opaque, `00` is invisible) and the others are red, green and blue. If you have a web colour such as `#2196F3`, write `0xFF` before the six digits.

Background, shape and border

Modifier	Effect
`background(color)`	Paints a rectangle behind the content.
`clip(shape)`	Cuts everything that is drawn after it to that shape.
`border(1.dp, color, shape)`	Draws a line along the edge of the shape.

The usual shapes are `RoundedCornerShape(12.dp)` (rounded corners), `CircleShape` and `RectangleShape`.

The **order** of modifiers matters here too. This is the usual order for a card, and the reason for each step:

```
Modifier
    .fillMaxWidth()                                  // 1. size
    .clip(RoundedCornerShape(12.dp))                 // 2. cut the corners
    .background(Color.White)                         // 3. paint inside
    .border(1.dp, Color.LightGray, RoundedCornerShape(12.dp))  // 4
    .padding(10.dp)                                  // 5. inner space
```

If `clip` came after `background`, the background would keep its square corners. If `padding` came before `background`, the space would stay outside the coloured area. Use the same shape in `clip` and in `border` so that the line follows the corners.

Text on a coloured background

Remember that `Text` accepts `color`, `fontSize` (in `sp`) and `fontWeight`. To put a text in the middle of a coloured area, the natural tool is a `Box` with `contentAlignment = Alignment.Center`:

```
Box(
    modifier = Modifier
        .height(90.dp)
        .clip(RoundedCornerShape(12.dp))
        .background(Color(0xFF2196F3)),
    contentAlignment = Alignment.Center
) {
    Text("Blue", color = Color.White, fontWeight = FontWeight.Bold)
}
```

2.9 Scrolling

A `Column` does not scroll by itself: what does not fit on the screen is cut off. To make it scroll, add `verticalScroll` to its modifier:

```
Column(
    modifier = Modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(12.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    // many children
}
```

Things to know:

- `rememberScrollState()` stores the scroll position. For now, use this pattern as it is; you will understand `remember` in Block 3.
- Write the `padding` **after** `verticalScroll`: then the space scrolls with the content. If it comes before, the space stays fixed at the edges of the screen.

- A scrolling column has **no limit of height**, so `Arrangement.Center`, `SpaceBetween` and the vertical `weight` have nothing to distribute and do not work as expected. Use `spacedBy` and fixed sizes.
- In the preview, add `showSystemUi = true` to see the screen with the bars of the phone, and run the app on the emulator to test the scroll: the preview does not scroll.
- `verticalScroll` creates **all** the children at once. It is perfect for a screen with a moderate number of elements. For long lists you will use `LazyColumn`, which only creates the visible items, in a later block.

> **Exercise 2.3**
> Create a `Column` with 25 `Text` (use a `repeat(25) { ... }` for it). Look at the preview and run it on the emulator: what happens at the bottom? Now add `verticalScroll(rememberScrollState())` and run it again.

2.10 Splitting the screen into composables

If you write the whole screen in a single function, it quickly becomes long and repetitive. When a piece of the design appears several times, or has its own meaning, create a composable for it. The data that changes between uses becomes a **parameter**:

```
@Composable
fun StatCard(title: String, value: Int, modifier: Modifier = Modifier) {
    Box(
        modifier = modifier
            .height(80.dp)
            .clip(RoundedCornerShape(12.dp))
            .background(Color(0xFF1565C0)),
        contentAlignment = Alignment.Center
    ) {
        Text("$title: $value", color = Color.White)
    }
}
```

The `modifier` parameter, with a default value, is the convention you saw in Block 1. It has a practical use here: **the caller decides how the composable fits in its parent**. As `weight` only exists inside a `Row` or a `Column`, the composable cannot use it itself, but the caller can pass it:

```
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    StatCard("Wins", 12, Modifier.weight(1f))
    StatCard("Draws", 3, Modifier.weight(1f))
    StatCard("Losses", 5, Modifier.weight(1f))
}
```

Inside `StatCard` the modifier received is the **first** of the chain (`modifier.height(80.dp)...`), so the composable keeps the modifiers its caller gave it and adds its own.

> **Tip**
> A composable that only paints something and always looks the same does not need a `modifier`; a composable that will be placed by its parent (inside a `Row`, for example) should have one.

2.11 Repeating UI from data

When a screen has many similar elements, such as a list of cards, you do not write one by one. You keep the data in a **list** and emit one composable for each item. As composables are Kotlin functions, you can use the loops and functions you already know:

```
val players = listOf(
    Triple("Ana", 20, Color(0xFFE91E63)),
    Triple("Luis", 22, Color(0xFF3F51B5)),
    Triple("Marta", 19, Color(0xFF4CAF50))
)

Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
    players.forEachIndexed { index, (name, age, color) ->
        PlayerRow(number = index + 1, name = name, age = age, color = color)
    }
}
```

Points of this code to understand:

- `Triple` groups three values. You could also use a `data class Player(...)`, which is clearer when the data has meaning: `players.forEach { player -> ... }`.
- `forEachIndexed` gives you the position (`index`) and the item. Positions start at 0, so we show `index + 1`.
- `(name, age, color)` is a **destructuring**: it unpacks the `Triple` into three variables.
- An `Int` must be converted to text to be shown in a `Text`: `number.toString()` or a string template such as `"$number"`.
- The list is defined **outside** the composable, as it does not change, so it is not created again on every redraw.

2.12 From design to code

When you have to build a screen from an image or a description, follow these steps:

1. **Draw the tree.** On paper, mark the vertical, horizontal and stacked areas. Give a name to each repeated piece.
2. **Write the skeleton** with the layouts and empty or fixed elements. Check the preview.
3. **Add size and space:** `fillMaxWidth`, `height`, `padding`, `spacedBy`.
4. **Add colour and shape:** `background`, `clip`, `border`, text styles.
5. **Extract the repeated pieces** into composables with parameters.

6. **Replace the repeated code with a loop** over a list of data.
7. **Run it** on the emulator or the device and test the scroll.

Work in small steps and check the preview after each one. If a layout does not behave as you expect, the **Layout Inspector** can help you: with the app running, open it from the tool windows of Android Studio (you can also search for it with Search Everywhere). It shows the tree of components of the screen and the size and position of each one.

> **Common mistakes**
>
> **Everything is stuck to the top-left corner:** you forgot `fillMaxSize()` or `contentAlignment`, or the layout is only as big as its content.
>
> **`weight` does not compile:** the composable is not a direct child of a `Row` or a `Column`.
>
> **The corners of the card are square:** the `clip` is after the `background`.
>
> **The content is cut at the bottom:** the column needs `verticalScroll`.
>
> **Arrangement does nothing in a column:** the column has no free height, or it is scrolling.

2.13 Working in the exercises project

The exercises of the module are all in one app, with a menu from which you open each exercise. For you, the important parts are:

Where	What it is
`Ejercicios/ej01/`, `ej02/`…	One folder per exercise. You write your solution there.
`fun Ejercicio01(modifier: Modifier = Modifier)`	The main composable of an exercise. The menu calls it, so keep its name and its `modifier` parameter.
`@Preview` at the end of the file	Lets you see the exercise without starting the app. Use `showSystemUi = true`.
`MenuEjercicios.kt`	The menu. Do not change it unless your teacher asks you to.

Put the small composables that you create (`StatCard` in the example) in the same file as the exercise, below the main composable. As in Block 1, work in small steps, check the preview and commit often.

2.14 Further practice and reading

Practise each exercise in a different Kotlin file, so that you keep all of them. Besides Exercise 2.4, you can use these official codelabs and documentation pages:

More practice

- “Create a basic layout”: a birthday card made with `Column` and `Text`.
- “Compose article”: a screen with an image, a title and two paragraphs.
- “Task manager”: a screen with an image and two texts, centred.

- "Compose quadrant": four areas of equal size, each one with a title and a text.

Documentation

- Basics of Compose layouts.
- Pixel density (to understand `dp` and `sp`).
- Text in Compose.
- Kotlin for Jetpack Compose.

You will find all of them in **developer.android.com/compose**.

Self-check questions

1. What is the difference between `Column`, `Row` and `Box`? Which one would you use to put a badge over an image?
2. What is the difference between `verticalArrangement` and `horizontalAlignment` in a `Column`?
3. Why is `Arrangement.Center` not visible in a `Column` that is only as big as its content?
4. What is the difference between `padding` and `Arrangement.spacedBy`? When do you use each one?
5. Three boxes have `weight(1f)`, `weight(1f)` and `weight(2f)` in a `Row`. How is the width shared?
6. Why does the order `clip`, `background`, `border`, `padding` make sense for a card?
7. What does the colour `Color(0xFF4CAF50)` mean? Which part is the transparency?
8. What do you need to add to a `Column` to make it scroll? Where should its `padding` go and why?
9. Why does a composable such as `StatCard` receive a `modifier` parameter instead of using `weight` inside?
10. How can you show 12 cards without writing 12 times the same code?
11. A `Box` contains another `Box` with a text. You use `contentAlignment = Alignment.Center` in the outer box. Is the text centred? Why?
12. When is it better to use `Spacer` than `spacedBy`? How do you push an element to the end of a `Row`?
13. How do you make a `Row` scroll horizontally? Write the modifier.

Key terms

English	Español	Section
Layout	Disposición / contenedor	2.1
Column / Row / Box	Columna / Fila / Caja	2.2–2.4
Main axis / cross axis	Eje principal / eje transversal	2.5
Arrangement	Distribución	2.5

English	Español	Section
Alignment	Alineación	2.5
Padding	Relleno / margen interior	2.6
Weight	Peso	2.6
Shape	Forma	2.8
Scroll	Desplazamiento	2.9
Destructuring	Desestructuración	2.11
Spacer	Espaciador	2.6